UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Software



CONWAY PARALLEL

Documentación de algoritmos

Simulación secuencial y paralela del Juego de la Vida

Programación Concurrente y Paralela

Grupo 01 | Periodo 2026-II

Integrantes

Bautista de la Cruz, Claudia	23200239
Bernal Hernández, Ryan	25200179
Ccahuana Quiñones, Valeria	23200248
Díaz Dionisio, Álvaro	11200142
Guzmán Chávez, Manuel	23200264
Hanampa Bellido, Iris	23200265
Munayco Sirlopu, Ronald	22200259
Rosales Trinidad, Jeanmarco	23200302

Lima, Perú — 2026

Algoritmo 1. Estado siguiente de una célula

NextCellState. Calcula el estado de una célula en la siguiente generación a partir de sus ocho vecinos. Aplica la regla B3/S23 y devuelve 1 si la célula estará viva o 0 si estará muerta. No modifica el tablero de entrada.

Algorithm 1: NextCellState

Data: *current*: `vector<uint8_t>`; *stride*, *row*, *col*: `size_t`
Result: *state*: `uint8_t`, with value 0 or 1

$i \leftarrow row \times stride + col$
$neighbors \leftarrow 0$
$neighbors \leftarrow neighbors + current[i - stride - 1]$
$neighbors \leftarrow neighbors + current[i - stride]$
$neighbors \leftarrow neighbors + current[i - stride + 1]$
$neighbors \leftarrow neighbors + current[i - 1]$
$neighbors \leftarrow neighbors + current[i + 1]$
$neighbors \leftarrow neighbors + current[i + stride - 1]$
$neighbors \leftarrow neighbors + current[i + stride]$
$neighbors \leftarrow neighbors + current[i + stride + 1]$
if $neighbors = 3$ ***or*** $(current[i] = 1$ ***and*** $neighbors = 2)$ **then**
  return 1
else
  return 0
end

Algoritmo 2. Evolución secuencial

SequentialStep. Calcula una generación completa con un único hilo. Recorre cada fila y columna, escribe los nuevos estados en el vector `next` y cuenta las células vivas. Al terminar, intercambia los buffers y actualiza la generación.

Algorithm 2: SequentialStep

Data: S: `GameState` (`struct`); $S.current$, $S.next$: `vector<uint8_t>`
Result: S: siguiente generación, población y contador actualizados

$nextAlive \leftarrow 0$
for $row \leftarrow 1$ **to** $S.height$ **do**
  for $col \leftarrow 1$ **to** $S.width$ **do**
    $i \leftarrow row \times S.stride + col$
    $state \leftarrow$ `NextCellState`$(S.current, S.stride, row, col)$
    $S.next[i] \leftarrow state$
    $nextAlive \leftarrow nextAlive + state$
  end
end
$S.aliveCount \leftarrow nextAlive$
`SwapBuffers`(S)
$S.generation \leftarrow S.generation + 1$

Algoritmo 3. Evolución paralela con OpenMP

OpenMPStep. Calcula una generación distribuyendo las filas entre varios hilos de OpenMP. Cada hilo lee el vector compartido `current`, escribe únicamente sus filas de `next` y acumula su población local. OpenMP combina los conteos mediante una reducción antes del intercambio de buffers.

Algorithm 3: OpenMPStep

Data: S: `GameState` (`struct`); $S.current$, $S.next$: `vector<uint8_t>`
Result: S: siguiente generación, población y contador actualizados

$nextAlive \leftarrow 0$
parallel region reduction$(+ : nextAlive)$ **do**
  omp for schedule(static) do
    for $row \leftarrow 1$ **to** $S.height$ **do**
      for $col \leftarrow 1$ **to** $S.width$ **do**
        $i \leftarrow row \times S.stride + col$
        $state \leftarrow$ `NextCellState`$(S.current, S.stride, row, col)$
        $S.next[i] \leftarrow state$
        $nextAlive \leftarrow nextAlive + state$
      end
    end
  end omp for
end parallel region
$S.aliveCount \leftarrow nextAlive$
`SwapBuffers`(S)
$S.generation \leftarrow S.generation + 1$

Algoritmo 4. Distribución de filas con MPI

DistributeRows. Divide el tablero en franjas de filas consecutivas. Si la altura no es divisible entre el número de procesos, los primeros ranks reciben una fila adicional. El rank 0 distribuye las franjas mediante `MPI_Scatterv`. Cada proceso copia las filas recibidas al interior de su tablero local y deja los halos en cero.

Algorithm 4: DistributeRows

Data: *cells*: `vector<uint8_t>` on rank 0; W, H: board width and height; r, P: rank and process count; *comm*: `MPI_Comm`

Result: S: `GameState` local con halos; $firstRow, rowCount$: primera fila global y cantidad de filas locales

$base \leftarrow \lfloor H/P \rfloor$
$extra \leftarrow H \bmod P$
for $i \leftarrow 0$ **to** $P-1$ **do**
  $rows_i \leftarrow base$
  if $i < extra$ **then**
    $rows_i \leftarrow rows_i + 1$
  end
  $start_i \leftarrow i \times base + \min(i, extra)$
  $counts[i] \leftarrow rows_i \times W$
  $displacements[i] \leftarrow start_i \times W$
end
$firstRow \leftarrow start_r$
$rowCount \leftarrow rows_r$
`MPI_Scatterv`$(cells, counts, displacements, localRows, counts[r], 0, comm)$
$(S.width, S.height, S.stride) \leftarrow (W, rowCount, W+2)$
$S.current, S.next \leftarrow$ ZeroVector$((rowCount+2) \times S.stride)$
$S.current[1 \ldots rowCount,\ 1 \ldots W] \leftarrow localRows$
return $S, firstRow, rowCount$

Algoritmo 5. Evolución paralela con MPI

MPIStep. Calcula una generación del tablero distribuido. Cada proceso con filas intercambia los bordes de su franja con los procesos vecinos y aplica `SequentialStep` sobre su tablero local. Al final, todos los procesos obtienen la población global mediante una reducción.

Algorithm 5: MPIStep

Data: S: `GameState` local con halos; $firstRow, rowCount$: límites de la franja local; H: altura global; r: rank local; $comm$: `MPI_Comm`
Result: S: estado local actualizado; $globalAlive$: población global

$valid \leftarrow$ IsCompatible$(S, firstRow, rowCount, H)$
$allValid \leftarrow$ `MPI_Allreduce`$(valid, \texttt{MPI_MIN}, comm)$
if $allValid = 0$ **then**
  raise `InvalidDistributedState`
end
$S.current[0,\ 0 \dots S.stride - 1] \leftarrow 0$
$S.current[rowCount + 1,\ 0 \dots S.stride - 1] \leftarrow 0$
if $rowCount > 0$ **then**
  $above \leftarrow$ `MPI_PROC_NULL`
  $below \leftarrow$ `MPI_PROC_NULL`
  if $firstRow > 0$ **then**
    $above \leftarrow r - 1$
  end
  if $firstRow + rowCount < H$ **then**
    $below \leftarrow r + 1$
  end
  `MPI_Sendrecv`$(\ S.current[1,\ 1 \dots S.width], above,$
   $S.current[rowCount + 1,\ 1 \dots S.width], below, 0, comm)$
  `MPI_Sendrecv`$(\ S.current[rowCount,\ 1 \dots S.width], below,$
   $S.current[0,\ 1 \dots S.width], above, 1, comm)$
end
SequentialStep(S)
$globalAlive \leftarrow$ `MPI_Allreduce`$(S.aliveCount, \texttt{MPI_SUM}, comm)$
return $globalAlive$

Algoritmo 6. Evolución paralela con CUDA

CUDAStep. Reinicia el contador de células vivas y lanza `EvolveKernel` en el mismo stream de CUDA. El host espera a que termine el kernel antes de intercambiar los punteros de los buffers e incrementar la generación.

Algorithm 6: CUDAStep

Data: S: estado del motor CUDA; $S.current$, $S.next$: buffers en GPU; $S.population$: contador en GPU de tipo `unsigned long long`; $S.launch$: geometría de lanzamiento; $S.stream$: `cudaStream_t`

Result: S: buffers intercambiados, población y generación actualizadas

1 CHECK(`cudaMemsetAsync`($S.population, 0, \text{sizeof}(\texttt{unsigned long long}), S.stream$))
2 $grid \leftarrow (S.launch.gridX, S.launch.gridY)$
3 $block \leftarrow (S.launch.block.x, S.launch.block.y)$
4 EVOLVEKERNEL$\langle\langle\langle grid, block, 0, S.stream\rangle\rangle\rangle$
5 $(S.current, S.next, S.width, S.height, S.stride, S.population)$
6 CHECK(`cudaGetLastError`())
7 CHECK(`cudaStreamSynchronize`($S.stream$))
8 $(S.current, S.next) \leftarrow (S.next, S.current)$
9 $S.generation \leftarrow S.generation + 1$

Algoritmo 7. Actualización de células en la GPU

EvolveKernel. Cada hilo calcula la fila y la columna que le corresponden. Si la posición está dentro del tablero, calcula el nuevo estado de la célula. Los hilos fuera del tablero aportan cero y también participan en la reducción. Al terminar, un hilo por bloque suma el subtotal al contador global mediante una operación atómica.

Algorithm 7: EvolveKernel

Data: *current*, *next*: buffers de `uint8_t` en GPU; *width*, *height*, *stride*: `size_t`; *population*: contador en GPU de tipo `unsigned long long`
Result: *next*: tablero de la siguiente generación; *population*: total de células vivas

$col \leftarrow blockIdx.x \times blockDim.x + threadIdx.x$
$row \leftarrow blockIdx.y \times blockDim.y + threadIdx.y$
$lane \leftarrow threadIdx.y \times blockDim.x + threadIdx.x$
$threads \leftarrow blockDim.x \times blockDim.y$
$alive \leftarrow 0$
if $row < height$ ***and*** $col < width$ **then**
    $alive \leftarrow$ `NextCellState`$(current, stride, row + 1, col + 1)$
    $i \leftarrow (row + 1) \times stride + col + 1$
    $next[i] \leftarrow alive$
end
shared `unsigned long long` $partial[256]$
$partial[lane] \leftarrow alive$
`__syncthreads()`
$offset \leftarrow \lfloor threads/2 \rfloor$
while $offset > 0$ **do**
    if $lane < offset$ **then**
        $partial[lane] \leftarrow partial[lane] + partial[lane + offset]$
    end
    `__syncthreads()`
    $offset \leftarrow \lfloor offset/2 \rfloor$
end
if $lane = 0$ **then**
    `atomicAdd`$(population, partial[0])$
end